Reinforcement Learning

Dynamic Programming

Sutton/Barto* Chapter 4

Michael Hahsler, SMU

With figures from Sutton/Barto*

*Sutton and Barto, Reinforcement Learning: An Introduction, 2nd edition, MIT Press, Cambridge, MA, 2018

Summary of Notation

General

X	capital letters: random variables
x, p	lower-case letters: realizations of random variables or scalar functions
$\boldsymbol{w}$	Bold lower-case letters: real-valued vectors (even if random variables)
$\boldsymbol{W}$	bold capitals: matrices
α	Greek letters: parameters (vectors if in bolt)
$\Pr\{X = x\}$	probability that a random variable X takes on the value x
$X \sim p$	random variable X selected from distribution$p(x) = \Pr\{X = x\}$
$\mathbb{E}[X]$	expectation of a random variable X, i.e., $\mathbb{E}[X] = \sum_x p(x)x$
$\text{argmax}_\text{a}\ f(a)$	a value of action a at which $f(a)$ takes its maximal value

MDP

s, s'	states
a	an action
r	a reward
$\mathcal{S}$	set of all (nonterminal) states, $\mathcal{S}^+$ are all states
$\mathcal{A}(s)$	set of all actions available in state s
γ	discount-rate parameter
t	discrete time step
T	final time step of an episode (a.k.a. horizon)
A_t	random variable for the action at time t
S_t	random variable for the state at time t
R_t	random variable for the reward at time t
$p(s', r \mid s, a)$	probability of transition to state s' and receiving reward r, from state s taking action a
	expected immediate reward from state s after action a
$\pi(a\|s)$	probability of taking action a in state s under stochastic policy π
$\pi(s)$	action taken in state s under deterministic policy π

Value Function

G_t	return (cumulative reward) following time t
$G_{t:h}$	return from t to h (discounted and corrected)
$v_\pi(s)$	value of state s under policy π (expected return)
$v_*(s)$	value of state s under the optimal policy
V, V_t	array estimates of state-value function v_π or v_*
$q_\pi(s, a)$	value of taking action a in state s under policy π
$q_*(s, a)$	value of taking action a in state s under the optimal policy
Q, Q_t	array estimates of action-value function q_π or v_*

Goal

- **Remember**: MDPs have an optimal deterministic policy.
- **Goal**: Compute the **optimal deterministic policy $\boldsymbol{\pi}_*$** given a perfect finite MDP model.
- Bellman optimality equations:

$$v_*(s) = \max_a \mathbb{E}_\pi \left[R_{t+1} + \gamma v_*(S_{t+1}) | S_t = s, A_t = a\right]$$

or

$$q_*(s,a) = \max_a \mathbb{E}_\pi \left[R_{t+1} + \gamma \underbrace{\max_{a'} q_*(S_{t+1}, a')}_{= v_*(S_{t+1})} \middle| S_t = s, A_t = a\right]$$

- **Issues**: The value estimate for S_t depends on the current estimates of the successor states S_{t+1}! We need to solve all optimality equations at the same time.

Tabular Methods

- Tabular means: The value functions v or q are stored in arrays (tables) called V or Q.
- Tabular methods can often find the exact solution (i.e., optimal value function and policy).

- **Issue**: Table size:
  - v has $|\mathcal{S}|$ entries
  - q has $|\mathcal{S}| \times |\mathcal{A}|$ entries

  this is only feasible for problems with a small discrete state/action spaces!

- We will later talk about approximate methods that can work with large state spaces.

Value Function

s	State Value $V(s)$
(1,1)	0.7453
(1,2)	0.8016
…	…

Q-Table

s	a	$Q(s,a)$
(1,1)	Up	0.7453
(1,1)	Right	0.6709
(1,1)	Down	0.7003
(1,1)	Left	0.7109
…	…	…

Prediction vs. Control

- We will always talk about two tasks:

1. **Prediction to evaluate a policy**: Estimate the value function for a policy. We can use this to compare two policies.

2. **Control**: Find the optimal policy. This requires to be able to evaluate policies.

Policy Evaluation

What is the value of a given policy?

Policy Evaluation (Prediction): Estimate v_π

- Compute the state-value function v_π for a given policy π.

- Recursive relationship:

$$\begin{aligned} v_\pi(s) &\stackrel{\text{def}}{=} \mathbb{E}_\pi[G_t | S_t = s] \\ &= \mathbb{E}_\pi\ [R_{t+1} + \gamma G_{t+1} | S_t = s] \\ &= \mathbb{E}_\pi\ [R_{t+1} + \gamma v_\pi(S_{t+1}) | S_t = s] \\ &= \sum_a \pi(a|s) \sum_{s',r} p(s', r\ | s, a)[r + \gamma v_\pi(s')] \end{aligned}$$

- $|\mathcal{S}|$ equations, one for each state s.

- **Guarantee**: The exists a unique v_π if $\gamma < 1$ or termination is guaranteed by π.

Solution Option 1: Linear Program

- Directly solve the system of $|\mathcal{S}|$ linear equations.

$$v_\pi(s_1) = \sum_a \pi(a|s_1) \sum_{s',r} p(s', r\ |s_1, a)[r + \gamma v_\pi(s')]$$
$$v_\pi(s_2) = \sum_a \pi(a|s_2) \sum_{s',r} p(s', r\ |s_2, a)[r + \gamma v_\pi(s')]$$

$$\ldots \quad \text{for all } s \in \mathcal{S}$$

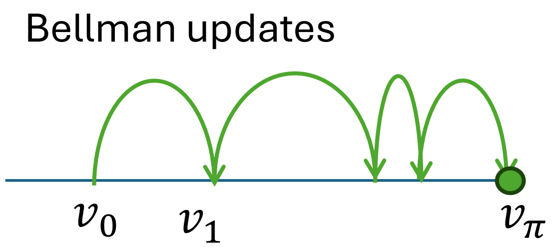
- This is hard if $|\mathcal{S}|$ is very large.

Solution Option 2: Iterative Policy Evaluation

Dynamic Programming: solves a complex optimization problem by breaking it down into smaller, overlapping subproblems and using their solutions to build the overall optimal solution.

- Start with arbitrary values for all $v(s)$
- Update all state values for k iterations (sweeps over the state space) using the **(expected) Bellman Update:**

$$v_{k+1}(s) \stackrel{\text{def}}{=} [R_{t+1} + \gamma v_k(S_{t+1}) | S_t = s]$$
$$= \underbrace{\sum_a \pi(a|s) \sum_{s',r} p(s',r\,|s,a)[r + \gamma v_k(s')]}_{\text{In DP called “expected update.” Uses the expectation over all possible actions and next states.}}$$



- **Convergence**: The Belman update is a contracting mapping with $v_k = v_\pi$ as its fixed point (for $0 \le \gamma < 1$).
I.e., updates converge to v_π for $k \to \infty$

Convergence of Iterative Policy Evaluation

- The expected Bellman update can be rewritten as an operator:
  Expected Bellman Operator T^π:

$$(T^\pi v)(s) = \sum_a \pi(a|s) \sum_{s',r} p(s', r\,|s, a)[r + \gamma v(s')]$$

- The operator T^π is γ-contracting:

$$\|T^\pi v_\pi - T^\pi v\|_\infty \leq \gamma \|v_\pi - v\|_\infty \quad \text{for some} \quad \gamma < 1$$

This means the largest difference (infinity norm) will be reduced, leading to convergence if T^π is applied many times.

Convergence is only guaranteed if the discount rate is strictly smaller than 1!

Alg. Iterative Policy Evaluation

Notation: V is the tabular estimate of v

Iterative Policy Evaluation, for estimating $V \approx v_\pi$

Input π, the policy to be evaluated
Algorithm parameter: a small threshold $\theta > 0$ determining accuracy of estimation
Initialize $V(s)$ arbitrarily, for $s \in \mathcal{S}$, and $V(\mathit{terminal})$ to 0

Loop:
  $\Delta \leftarrow 0$
  Loop for each $s \in \mathcal{S}$:
    $v \leftarrow V(s)$
    $V(s) \leftarrow \sum_a \pi(a|s) \sum_{s',r} p(s',r|s,a)\big[r + \gamma V(s')\big]$
    $\Delta \leftarrow \max(\Delta, |v - V(s)|)$
until $\Delta < \theta$

Bellman operator T^π:
$V_{k+1} = T^\pi V_k$

Stop when value function changes little:
$\|T^\pi V_k - V_k\|_\infty < \theta$
It can be shown that the error is bounded:
$\|V_k - v_\pi\|_\infty < \dfrac{\theta}{1-\gamma}$

If we stop policy evaluation early (after n iterations) then it is called **modified policy iteration.**

Policy Iteration

Contol: Find the optimal policy.

Policy Improvement

- **Goal**: Find a better policy.
- **Policy improvement theorem**: If

$$q_\pi(s, \pi'(s)) \geq v_\pi(s) \;\; \forall s \in \mathcal{S}$$

then π' is a least as good or better than π, i.e.,

$$v_{\pi'}(s) \geq v_\pi(s) \;\; \forall s \in \mathcal{S}.$$

- **Policy improvement**: create a new policy π' by choosing in each state the best (aka greedy) action using $q_\pi(s, a)$:

$$\pi'(s) = \operatorname*{argmax}_a q_\pi(s, a)$$

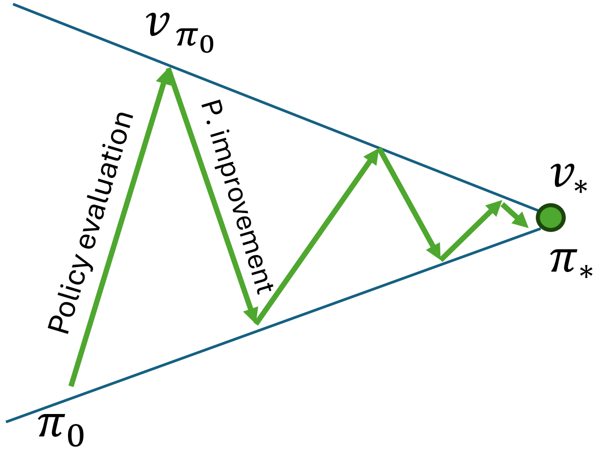
- **Guarantee**: This gives a strictly better policy π' unless π is already optimal.

Policy Iteration

- Sequence of monotonically improving policies:

$$\pi_0 \xrightarrow{E} v_{\pi_o} \xrightarrow{I} \pi_0 \xrightarrow{E} v_{\pi_o} \xrightarrow{I} \pi_1 \xrightarrow{E} v_{\pi_1} \xrightarrow{I} \pi_2 \xrightarrow{E} v_{\pi_2} \xrightarrow{I} \dots \pi_* \xrightarrow{E} v_*$$

$\xrightarrow{E}$ policy eval. step
$\xrightarrow{I}$ policy improvement step



Guarantees:

- Policy improvement is guaranteed to improve the policy + finite MDPs have a finite number of policies → Policy Iteration must **converge**!

Notes:

- Policy becomes stable before v_* is reached.
- Policy iteration typically converges very fast (a few iterations)
- **Issue**: Policy estimation is computationaly expensive! Using fewer iterations leads to modified policy iteration.

Alg. Policy Iteration

Policy Iteration (using iterative policy evaluation) for estimating $\pi \approx \pi_*$

1. Initialization
   $V(s) \in \mathbb{R}$ and $\pi(s) \in \mathcal{A}(s)$ arbitrarily for all $s \in \mathcal{S}$; $V(\mathit{terminal}) \doteq 0$

2. Policy Evaluation
   Loop:
   $\quad\Delta \leftarrow 0$
   $\quad$Loop for each $s \in \mathcal{S}$:
   $\qquad v \leftarrow V(s)$
   $\qquad V(s) \leftarrow \sum_{s',r} p(s',r|s,\pi(s))\big[r + \gamma V(s')\big]$
   $\qquad \Delta \leftarrow \max(\Delta, |v - V(s)|)$
   until $\Delta < \theta$ (a small positive number determining the accuracy of estimation)

3. Policy Improvement
   policy-stable $\leftarrow$ *true*
   For each $s \in \mathcal{S}$:
   $\quad$*old-action* $\leftarrow \pi(s)$
   $\quad \pi(s) \leftarrow \arg\max_a \sum_{s',r} p(s',r|s,a)\big[r + \gamma V(s')\big]$
   $\quad$If *old-action* $\neq \pi(s)$, then *policy-stable* $\leftarrow$ *false*
   If *policy-stable*, then stop and return $V \approx v_*$ and $\pi \approx \pi_*$; else go to 2

Just policy evaluation from before.
Can also stop this loop early
(**modified policy iteration**).

Picks the greedy action
with the highest Q-value.

Value Iteration

Control: Find the optimal value function.

Value Iteration

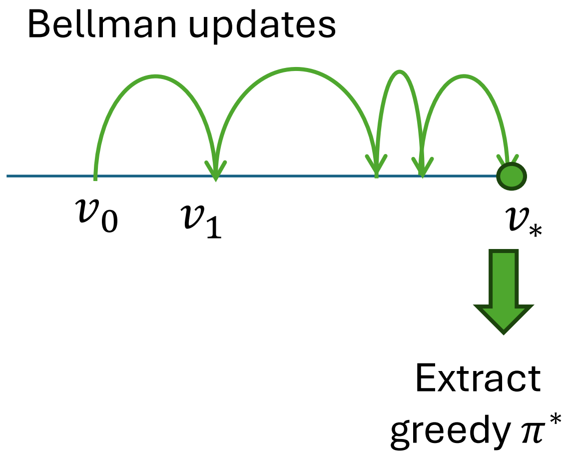
- Combines the policy improvement and truncated policy evaluation steps. This is called the **expected Bellman optimality update**.

$$v_{k+1}(s) \overset{\text{def}}{=} \max_a \mathbb{E}[R_{t+1} + \gamma v_k(S_{t+1}) | S_t = s, A_t = a]$$
$$= \max_a \sum_{s',r} p(s', r \,| s, a)[r + \gamma v_k(s')]$$

$\max_a$ ... policy improvement
$\mathbb{E}$ ... policy evaluation

- Written as the **Bellman optimality operator**:

$$(T^* v)(s) = \max_a \sum_{s',r} p(s', r \,| s, a)[r + \gamma v(s')]$$



- **Guarantee:** T^* is also γ-contracting. The Sequence $\{v_k\}$ converges to v_*

Alg. Value Iteration

Policy Iteration (using iterative policy evaluation) for estimating $\pi \approx \pi_*$

1. Initialization
   $V(s) \in \mathbb{R}$ and $\pi(s) \in \mathcal{A}(s)$ arbitrarily for all $s \in \mathcal{S}$; $V(terminal) \doteq 0$

2. Policy Evaluation
   Loop:
   $\Delta \leftarrow 0$
   Loop for each $s \in \mathcal{S}$:
   $v \leftarrow V(s)$
   $\boxed{V(s) \leftarrow \sum_{s',r} p(s',r|s,\pi(s))[r + \gamma V(}$
   $\Delta \leftarrow \max(\Delta, |v - V(s)|)$
   until $\Delta <$

3. Policy In
   policy-st
   For each
   old-a
   $\pi(s) \leftarrow \arg\boxed{\max_a \sum_{s',r} p(s',r|s,a)[r + \gamma}$
   If *old-action* $\neq \pi(s)$, then *policy-stable* $\leftarrow$
   If *policy-stable*, then stop and return $V \approx v_*$

Combine improvement and modified evaluation (1 iteration) into a single update.

Value Iteration, for estimating $\pi \approx \pi_*$

Algorithm parameter: a small threshold $\theta > 0$ determining accuracy of estimation
Initialize $V(s)$, for all $s \in \mathcal{S}^+$, arbitrarily except that $V(terminal) = 0$

Loop:
$\Delta \leftarrow 0$
Loop for each $s \in \mathcal{S}$:
$v \leftarrow V(s)$
$\boxed{V(s) \leftarrow \max_a \sum_{s',r} p(s',r|s,a)[r + \gamma V(s')]}$
$\Delta \leftarrow \max(\Delta, |v - V(s)|)$
until $\Delta < \theta$

Output a deterministic policy, $\pi \approx \pi_*$, such that
$\pi(s) = \arg\max_a \sum_{s',r} p(s',r|s,a)[r + \gamma V(s')]$

Bellman optimality operator:
$V_{k+1} = T^* V_k$

Extract the greedy policy.

Issues with DP Methods

Space

- Tabular method: stores the complete value function as table V.

Time

- Updates repeatedly sweep over the whole state space.
- Wastes computation by updating states that are not relevant for the optimal behavior.

GPI: Generalized Policy Iteration

GPI: Generalized Policy Iteration

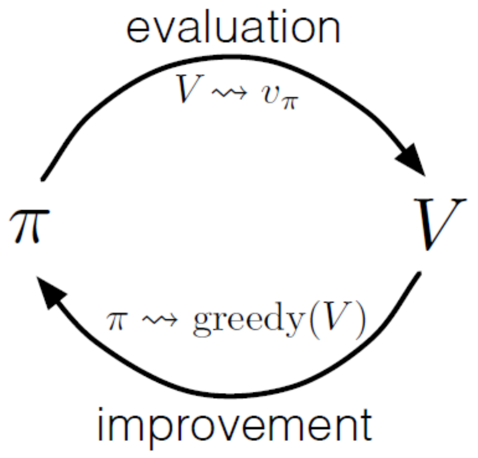
Steps

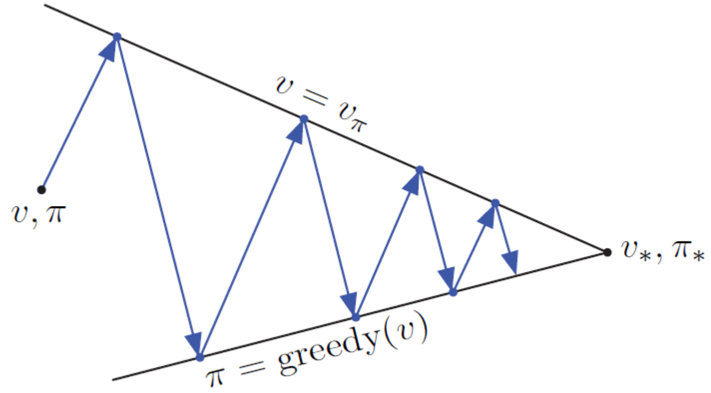
- **Policy evaluation**: make the value function consistent with the current policy π.
- **Policy improvement**: make policy greedy with respect to the current value function estimate V.

Most DP methods can be described as a special case of GPI:

- **Policy iteration**: each eval and improvement step is completed (by a sweep over $\mathcal{S}$) before moving on.
- **Value iteration**: uses a single iteration for the policy iteration.
- **Asynchronous DP**: Instead of sweeps over $\mathcal{S}$, updates states in some other order. This interleaves pol. eval. and pol. Improvement.

All three approaches typically converge to the optimal solution.





Efficiency

- **Direct Search**: Search space $O(|A|^{|\mathcal{S}|})$ is **exponential** in the number of states $|\mathcal{S}|$ and actions $|\mathcal{A}|$.

- **LP (linear program):** Solve a system of $|\mathcal{S}|$ equations with $|\mathcal{S}|$ state variables and $|\mathcal{S}| \times |\mathcal{A}|$ constraints has a time complexity of $O(|\mathcal{S}|^3)$.

- **DP**: Time $O(|\mathcal{S}|^2 \times |\mathcal{A}|)$ is **polynomial** in $|\mathcal{S}|$ and $|\mathcal{A}|$.

Search and LP are impractical. **DP can be used today with millions of states.**

What you need to know

- Requirements:
  - **Model access**: DP methods need access to the **model**'s function p.
  - **Small discrete state/action space** since it is a tabular method.

- Tasks:
  - **Prediction**: Calculates the value function for a given policy.
  - **Control**: Calculates the optimal policy for an MDP.

- **Approach**: Convert the recursive Bellman equations into the Belmann update operators and sweep the state space till the optimal solution is found.
  - Expected Bellman update operator T^{π} vs. Bellman optimality operator T^{*}
  - Value iteration is a special case of modified policy iteration.
  - Why and when is convergence guaranteed?

- **GPI**: All DP methods can be generalized as the GPI approach.